Subject: ASD&D	**Semester:** V
Class: TE	**Batch:** T11
Name: Ananya Amesur	**Roll No:** 2401005

Experiment 4

Aim: To set up a Continuous Integration Pipeline using Jenkins/GitHub Act.

Lab Outcome: Mapped LO3

LO3: Build and manage automated CI/CD pipelines for application development, testing, packaging and deployment.

Theory:

- Continuous Integration (CI) & Continuous Delivery (CD)

➢ Continuous Integration (CI): A modern DevOps development practice where developers merge their code changes frequently into a central shared repository (such as GitHub). Every commit triggers an automated build and execution of tests.

  - Primary Objective: Detect integration bugs and software defects early in the software development lifecycle (SDLC), preventing "integration hell" when multiple developers work simultaneously.

➢ Continuous Delivery / Continuous Deployment (CD): An extension of CI where code changes are automatically prepared (Delivery) or deployed directly (Deployment) to testing or production environments once all continuous integration builds pass successfully.

- Overview of Jenkins

➢ Jenkins Server: An open-source automation server written in Java that serves as the foundation for CI/CD pipelines.

➢ Core Responsibilities:

  - Automation: Automates repetitive engineering tasks, including code compilation, unit testing, static code analysis, artifact generation, and deployment.

- Plugin Ecosystem: Supports thousands of plugins to integrate seamlessly with version control systems (Git, SVN), build tools (Maven, Gradle, npm), and deployment platforms (Docker, Kubernetes, AWS).
- Event Triggers: Can trigger workflows automatically via Git webhooks, scheduled cron jobs, or manually through its Web User Interface (UI).

- Jenkins Architecture

➢ Jenkins Master (Controller): Handles configuration, schedules jobs, monitors build executors, dispatches builds to agents, and serves the Web UI.

➢ Jenkins Agents (Nodes/Workers): External machines or containers that execute the actual build steps dispatched by the Controller, allowing distributed execution and scaling.

- Types of Jenkins Pipelines

A Jenkins Pipeline is a suite of plugins that supports implementing and integrating continuous delivery pipelines into Jenkins using code (often stored in a Jenkinsfile). There are two main types of pipeline syntaxes:

A. Declarative Pipeline

➢ Syntax Format: Enclosed inside a pipeline {} block.

➢ Characteristics:

- Strict, predefined structure designed to be simple, highly readable, and maintainable for standard CI/CD workflows.
- Offers superior syntax validation and error checking directly within the Jenkins UI prior to execution.
- Organized strictly into defined sections such as agent, stages, stage, and steps.

B. Scripted Pipeline

➢ Syntax Format: Enclosed inside a node {} block.

➢ Characteristics:

- Built using full Apache Groovy scripting capability, offering maximum procedural flexibility and control.

- Ideal for advanced, dynamic scenarios requiring complex control loops, conditional execution, or specialized logic.
- Requires a higher learning curve due to its unstructured, code-heavy nature.

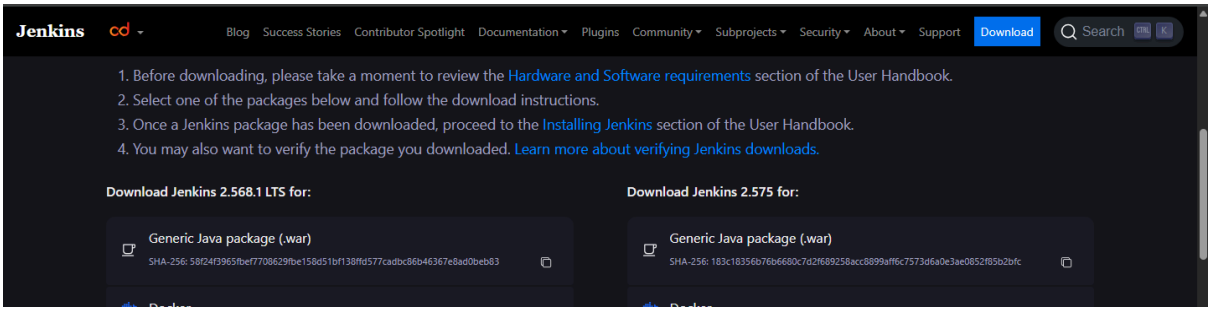
Execution:

Step 1: Java (JDK) Installation Verification

```
C:\Users\LENOVO>java -version
java version "22.0.2" 2024-07-16
Java(TM) SE Runtime Environment (build 22.0.2+9-70)
Java HotSpot(TM) 64-Bit Server VM (build 22.0.2+9-70, mixed mode, sharing)
```

Step 2: Jenkins Server Launch

Downloaded the Generic Java package (jenkins.war) from the official website.

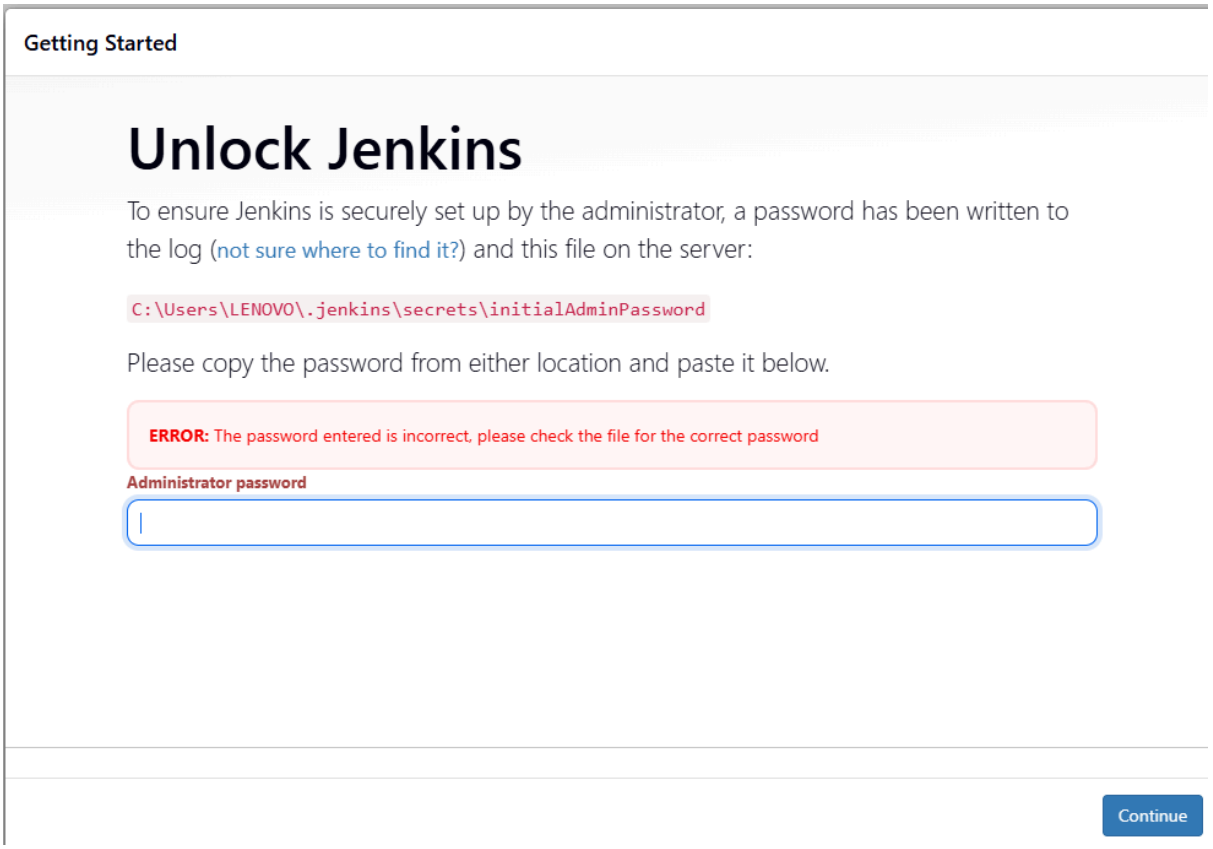


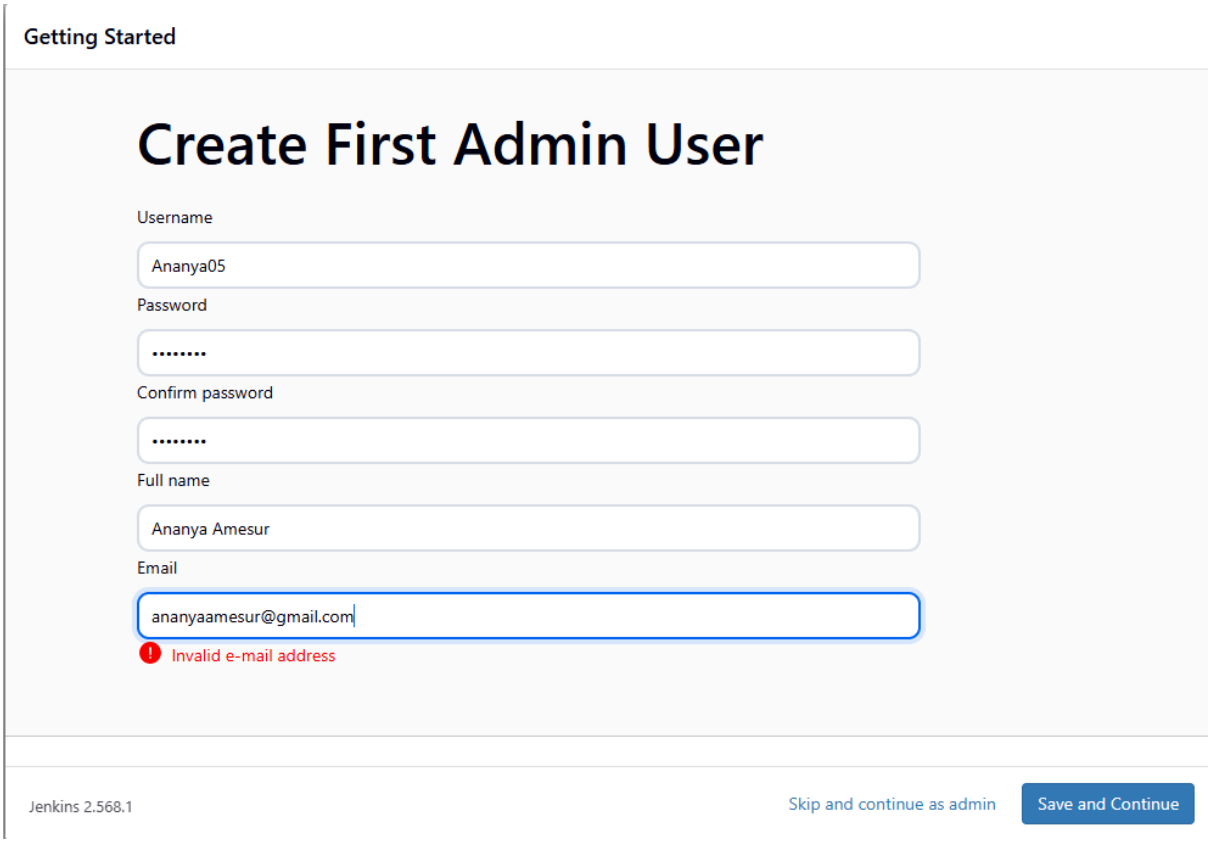
Executed Jenkins using the --enable-future-java flag to support JDK 22:

```
C:\Users\LENOVO\Downloads>java -jar jenkins.war --enable-future-java
Running with Java 22 from C:\Program Files\Java\jdk-22, which is not fully supported. Continuing because --enable-future-java is set. Supported Java version
s are: [21, 25]. See https://jenkins.io/redirect/java-support/ for more information.
Running from: C:\Users\LENOVO\Downloads\jenkins.war
webroot: C:\Users\LENOVO\.jenkins\war
2026-08-03 10:09:48.162+0000 [id=1]     INFO    winstone.Logger#logInternal: Beginning extraction from war file
2026-08-03 10:09:48.209+0000 [id=1]     WARNING o.e.j.ee9.nested.ContextHandler#setContextPath: Empty contextPath
2026-08-03 10:09:48.287+0000 [id=1]     INFO    org.eclipse.jetty.server.Server#doStart: jetty-12.1.8; built: 2026-04-01T01:09:18.991Z; git: c9cdc9aaa434a36
65b8a53b4d1cc3684992da649; jvm 22.0.2+9-70
2026-08-03 10:09:48.881+0000 [id=1]     INFO    o.e.j.e.w.StandardDescriptorProcessor#visitServlet: NO JSP Support for /, did not find org.eclipse.jetty.ee9
.jsp.JettyJspServlet
2026-08-03 10:09:48.927+0000 [id=1]     INFO    o.e.j.s.DefaultSessionIdManager#doStart: Session workerName=node0
2026-08-03 10:09:49.224+0000 [id=1]     INFO    hudson.WebAppMain#contextInitialized: Jenkins home directory: C:\Users\LENOVO\.jenkins found at: $user.home/
.jenkins
2026-08-03 10:09:49.301+0000 [id=1]     INFO    o.e.j.s.handler.ContextHandler#doStart: Started oeje9n.ContextHandler$CoreContextHandler@78d39a69{Jenkins v2
.568.1,/,b=file:///C:/Users/LENOVO/.jenkins/war/,a=AVAILABLE,h=oeje9n.ContextHandler$CoreContextHandler$CoreToNestedHandler@3c818ac4{STARTED}}
2026-08-03 10:09:49.316+0000 [id=1]     INFO    o.e.j.server.AbstractConnector#doStart: Started oejs.ServerConnector@41e0443d{HTTP/1.1, (http/1.1)}{0.0.0.0:
8080}
2026-08-03 10:09:49.316+0000 [id=1]     INFO    org.eclipse.jetty.server.Server#doStart: Started oejs.Server@3ae66c85{STARTING}[12.1.8,sto=0] @1029ms
2026-08-03 10:09:49.332+0000 [id=42]    INFO    winstone.Logger#logInternal: Winstone Servlet Engine running: controlPort=disabled
2026-08-03 10:09:49.441+0000 [id=41]    INFO    jenkins.model.Jenkins#<init>: Starting version 2.568.1
2026-08-03 10:09:49.522+0000 [id=51]    INFO    jenkins.InitReactorRunner$1#onAttained: Started initialization
2026-08-03 10:09:49.559+0000 [id=71]    INFO    jenkins.InitReactorRunner$1#onAttained: Listed all plugins
2026-08-03 10:09:50.293+0000 [id=63]    INFO    jenkins.InitReactorRunner$1#onAttained: Prepared all plugins
2026-08-03 10:09:50.293+0000 [id=56]    INFO    jenkins.InitReactorRunner$1#onAttained: Started all plugins
2026-08-03 10:09:50.293+0000 [id=56]    INFO    jenkins.InitReactorRunner$1#onAttained: Augmented all extensions
2026-08-03 10:09:50.497+0000 [id=52]    INFO    jenkins.InitReactorRunner$1#onAttained: System config loaded
2026-08-03 10:09:50.506+0000 [id=59]    INFO    jenkins.InitReactorRunner$1#onAttained: System config adapted
2026-08-03 10:09:50.506+0000 [id=53]    INFO    jenkins.InitReactorRunner$1#onAttained: Loaded all jobs
2026-08-03 10:09:50.507+0000 [id=53]    INFO    jenkins.InitReactorRunner$1#onAttained: Configuration for all jobs updated
2026-08-03 10:09:50.524+0000 [id=53]    INFO    jenkins.install.SetupWizard#init:
```

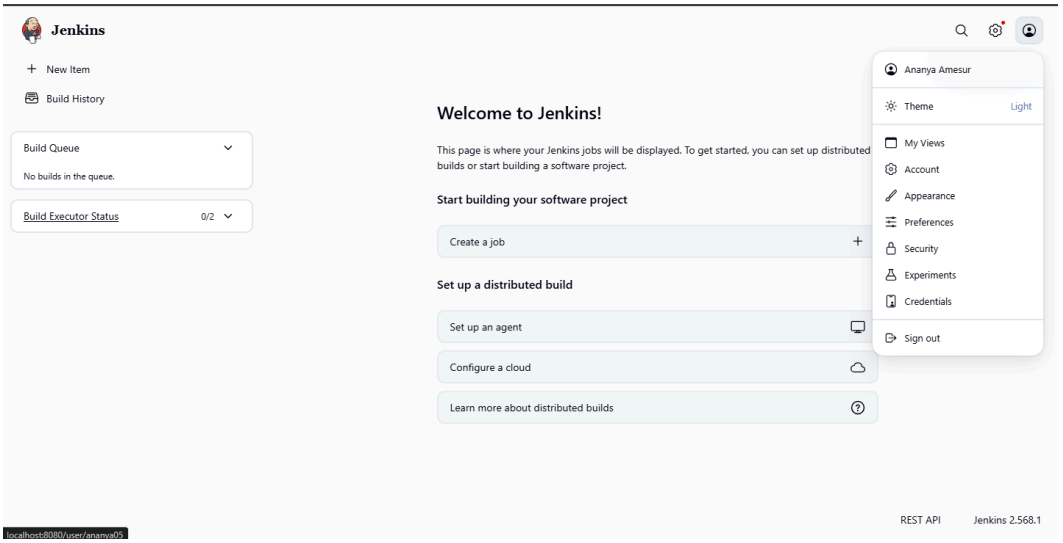
Step 3: Web Setup & Admin Configuration

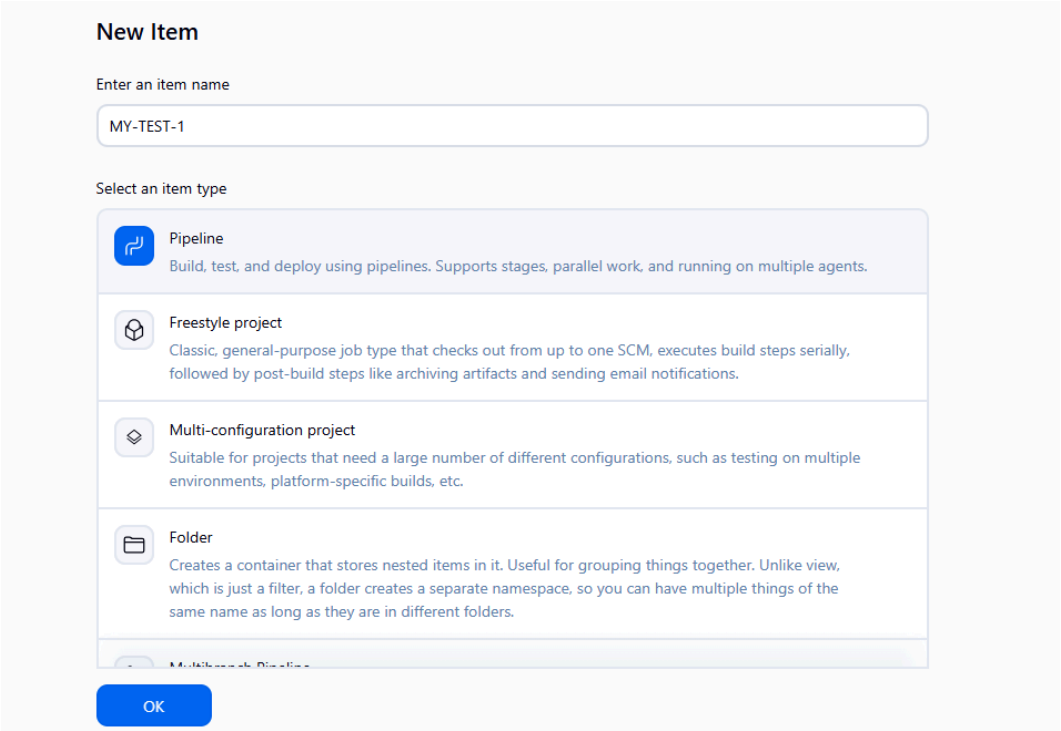
```
[CRLF]>
[CRLF]> ************************************************************
[CRLF]> ************************************************************
[CRLF]> ************************************************************
[CRLF]>
[CRLF]> Jenkins initial setup is required. An admin user has been created and a password generated.
[CRLF]> Please use the following password to proceed to installation:
[CRLF]>
[CRLF]> f87b65a1b4344b2da95b74b2782bc8b7
[CRLF]>
[CRLF]> This may also be found at: C:\Users\LENOVO\.jenkins\secrets\initialAdminPassword
[CRLF]>
[CRLF]> ************************************************************
[CRLF]> ************************************************************
[CRLF]> ************************************************************
```





Step 4: Creating Pipelines

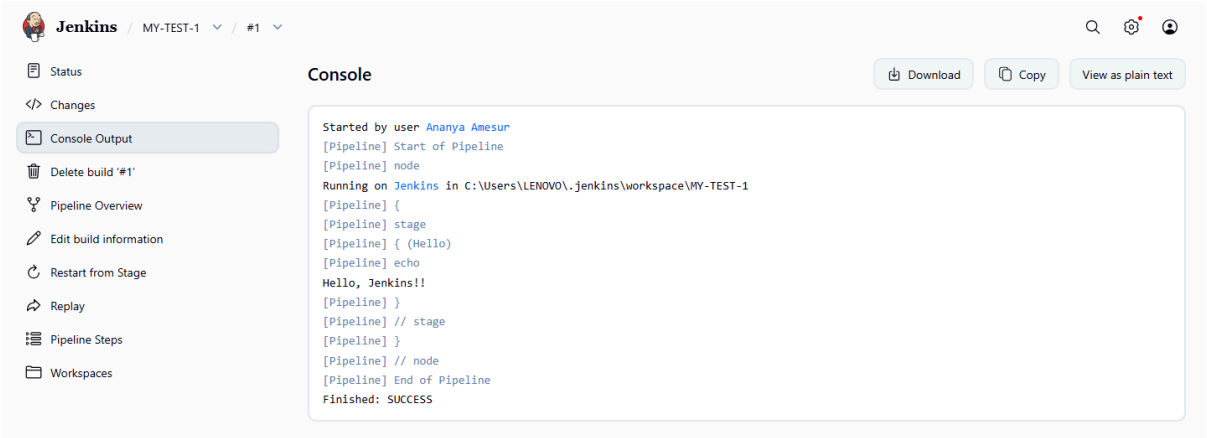




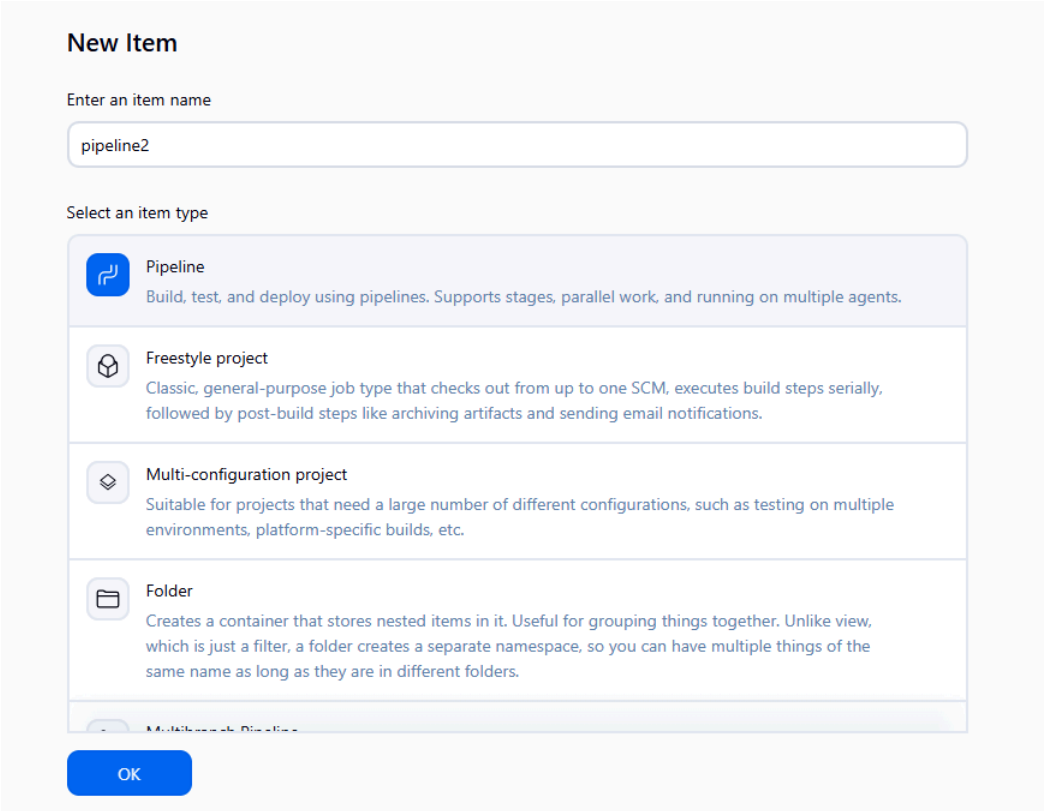
Code:

```
pipeline {
    agent any
    stages {
        stage('Hello') {
            steps {
                echo 'Hello, Jenkins!!'
            }
        }
    }
}
```

Output:



Step 5: Creating Scripted Pipeline



Code:

```
node {
    stage('Hello') {
        echo 'Hello, Scripted Jenkins Pipeline!'
    }
}
```

Output:



Conclusion:

Through this experiment, we learned the fundamentals of Continuous Integration (CI) and how Jenkins automates build and deployment workflows to eliminate manual integration errors. We gained hands-on experience verifying Java environments and deploying a standalone Jenkins server via the command-line interface. Additionally, we explored both Declarative and Scripted pipeline syntaxes, understanding how to structure automated build stages using code. By monitoring the console logs, we successfully executed and verified both pipeline builds. Overall, this experiment demonstrated how Jenkins serves as a crucial CI/CD automation tool.